

FECAP - Fundação Escola de Comércio Álvares Penteado
Curso Ciência da Computação – 4ºSemestre

Design de Software e Diagrama de UML

Entrega 2 – Engenharia de Software e Arquitetura de Sistemas

Caroliny Rossi Bittencourt – 24025959
Rafael Alves dos Santos Guimarães – 24025724
Duda Lucena Miguel – 24025889
Isadora Teixeira Santoma - 24026458

SÃO PAULO - 2025

Introdução

O design de software é a etapa de planejamento da estrutura e organização do sistema antes da implementação. Nela são definidas a arquitetura, os componentes e a forma como eles irão interagir, garantindo um código mais claro, reutilizável, seguro e de fácil manutenção. Para representar esse planejamento de maneira visual, utiliza-se a UML (Unified Modeling Language), um conjunto de diagramas que descrevem a estrutura e o comportamento do sistema. Dessa forma, o design de software e a UML facilitam a comunicação entre a equipe e orientam o desenvolvimento de forma mais organizada e consistente.

Design de software

Arquitetura de camadas

Camada de Apresentação

Dashboard interativo com duas visualizações: **CEO** e **CFO**.

- **CEO:** Comportamento dos usuários (quantidade de resgates, perfil por sexo/idade/localização e lojas mais acessadas), com filtros e gráficos interativos.
- **CFO:** Visão financeira baseada nos dados expostos pela API (ex.: valor de cupons e repasse para a plataforma), com filtros e gráficos para análise de desempenho.

Camada de Lógica de Negócios (API/Servidor):

Interpreta os filtros escolhidos pelo usuário e os converte em consultas adequadas ao banco de dados. Consolida e organiza os resultados para os gráficos/indicadores do dashboard, assegurando consistência das informações e bom desempenho durante a navegação.

Camada de Acesso a Dados (DAO/Models):

Comunica-se diretamente com o banco de dados, executando consultas parametrizadas de acordo com os filtros recebidos, com ordenação controlada e

paginação por cursor. Retorna os registros de forma estruturada para a camada de lógica.

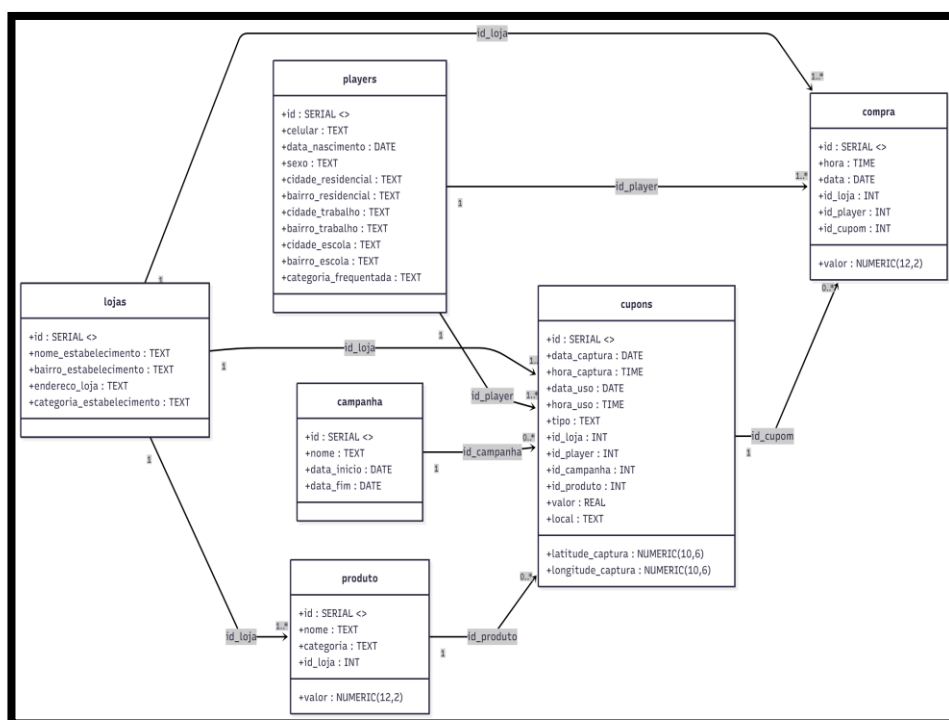
Banco de Dados:

Armazena de forma organizada as informações utilizadas no dashboard, incluindo as tabelas da Base Cadastral dos Players, Massa de Testes, Base Simulada de Pedestres, Base de Transações e Cupons Capturados. Permite consultas, filtragens e correlações que fundamentam as análises exibidas na interface, garantindo persistência, consistência e integridade dos dados.

Diagramas de UML

Diagrama de Classes

O Diagrama de Classes descreve a estrutura estática do sistema, evidenciando quais são as entidades principais do domínio, os atributos que cada uma armazena e como elas se relacionam entre si. Diferente de diagramas que mostram a interação ou o fluxo de execução, o Diagrama de Classes representa a base conceitual sobre a qual o sistema é construído, servindo como referência para o banco de dados, API e regras de negócio.



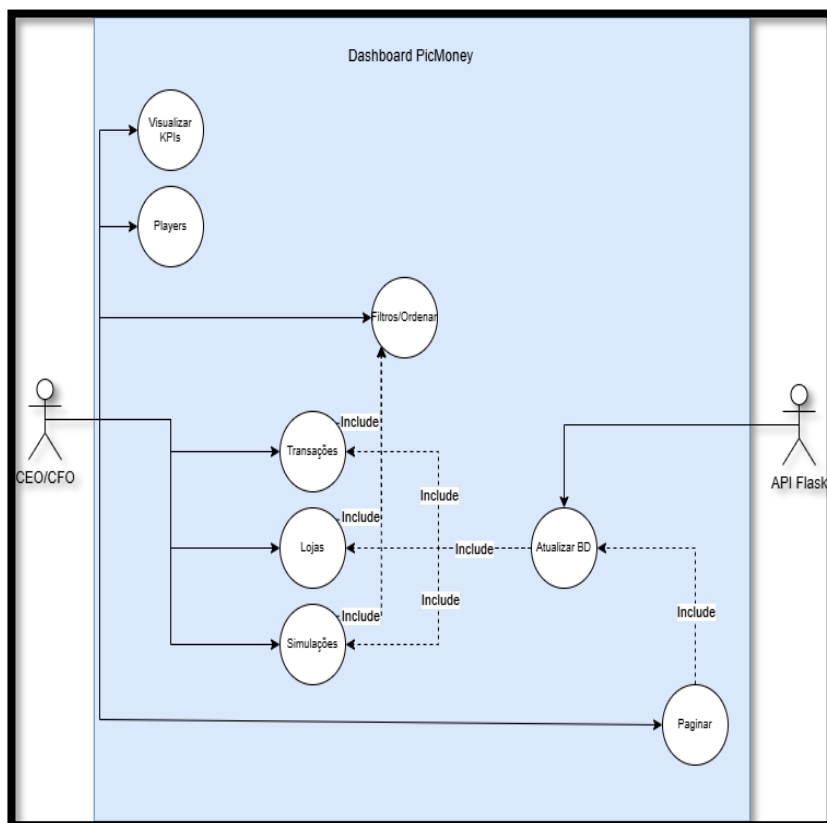
1 Imagem – Diagrama de Classes

As classes centrais são: **players** (usuários), **lojas** (estabelecimentos parceiros), **produto**, **campanha**, **cupons** e **compra**. Cada uma reúne informações específicas relevantes, como dados demográficos do usuário, localização da loja, valor do produto ou registros de captura e uso de cupons. Já os relacionamentos refletem a lógica do domínio: uma **loja** pode ter vários **produtos** e **cupons**, enquanto um **player** pode possuir diversos **cupons** e realizar múltiplas **compras**. A associação com **cupom** em **compra** é opcional, pois nem toda transação envolve desconto, assim como a ligação entre cupom e campanha/produto também pode ser facultativa.

Esse modelo torna a estrutura dos dados clara e organizada, permitindo que a API recupere informações de forma coerente e que a Dashboard apresente indicadores consistentes tanto para a visão do **CEO** (análises de comportamento de uso) quanto do **CFO** (análises financeiras e de desempenho de parceiros).

Diagrama de Casos de Uso

O Diagrama de casos de uso representa as interações entre os usuários e o sistema, destacando as principais funcionalidades da DashboardPicMoney. O ator principal é o usuário (CEO ou CFO), que acessa a interface para visualizar indicadores, aplicar filtros, consultar informações e analisar dados financeiros e comportamentais. O ator secundário é o **Serviço de API (Flask)**, responsável por processar as requisições e acessar o banco de dados SQLite, retornando os resultados para exibição.



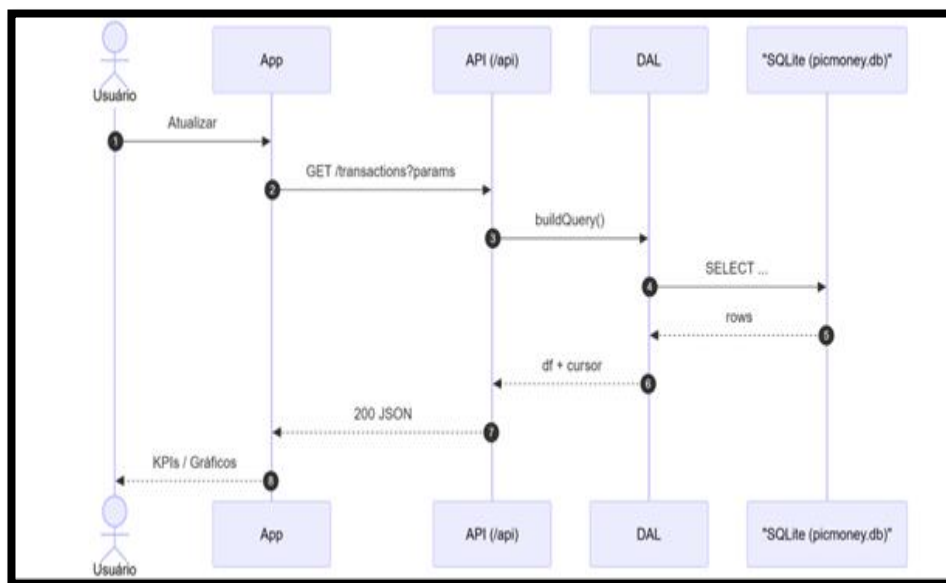
2 Imagem – Diagrama de Casos de Uso

O diagrama como mostra na Imagem 1 evidencia casos como visualizar dados, aplicar filtros, navegar entre páginas e gerar relatórios. As relações do tipo <<include>> indicam que certas ações dependem de outras, como a consulta que inclui filtragem e paginação para refinar os resultados.

Esse diagrama descreve o **comportamento externo do sistema**, mostrando o **que** o usuário pode fazer sem detalhar **como** essas funções são implementadas. Ele serve como base para a elaboração de diagramas complementares, como o de sequência e o de classes, facilitando o planejamento e a comunicação dentro da equipe de desenvolvimento.

Diagrama de Sequência

O Diagrama de Sequência ilustra a troca de mensagens entre os componentes do sistema ao longo do tempo, mostrando como ocorre o processo de atualização dos dados na Dashboard a partir da interação do usuário.



3 Imagem – Diagrama de Sequência

No diagrama de sequência da Imagem 3, o fluxo inicia quando o usuário (CEO/CFO) ajusta os filtros e solicita a atualização dos indicadores na interface desenvolvida em **Streamlit**. A aplicação então envia uma requisição para a **API Flask**, que funciona como camada intermediária entre o front-end e o banco de dados.

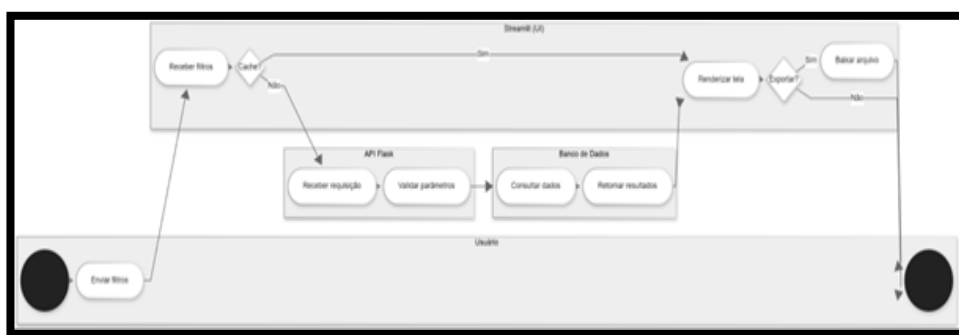
A API encaminha os parâmetros recebidos ao **Repositório (DAL)**, responsável por construir e executar a consulta SQL no banco **SQLite**. Após recuperar os dados filtrados, o repositório retorna os resultados à API, que os envia de volta ao Streamlit em formato

JSON. A interface utiliza essas informações para atualizar automaticamente KPIs, gráficos e tabelas exibidas ao usuário.

Esse diagrama é importante pois evidencia a sequência lógica das chamadas internas do sistema, complementando o Diagrama de Casos de Uso: enquanto o caso de uso descreve *o que* o sistema faz, o diagrama de sequência detalha *como* cada funcionalidade é executada passo a passo.

Diagrama de Atividades

O Diagrama de Atividades representa o fluxo de ações realizadas durante a atualização e visualização da Dashboard PicMoney. Ele descreve, de forma sequencial, como o usuário interage com o sistema e como os dados são processados até a exibição dos resultados.



4 Imagem – Diagrama de Atividades

Observado no diagrama de atividades na Imagem 2, o fluxo inicia quando o usuário define os filtros na interface (Streamlit). Em seguida, o sistema verifica se existe um cache válido para evitar consultas desnecessárias. Caso o cache esteja disponível, os dados são carregados imediatamente e os indicadores são atualizados na tela.

Se não houver cache, a interface aciona a API Flask, que valida os parâmetros e realiza a consulta ao banco de dados SQLite. Os resultados obtidos retornam para o Streamlit, que renderiza os KPIs e gráficos atualizados.

Ao final, o usuário pode optar por exportar os dados. Caso essa ação seja escolhida, o sistema gera o arquivo (CSV ou PNG) para download; caso contrário, o fluxo se encerra na exibição dos resultados na dashboard.

Assim, o diagrama evidencia o ciclo completo de interação, desde o acionamento dos filtros até a entrega das informações ao usuário, incluindo decisões, processamento e finalização do processo.

Conclusão

A aplicação do design de software e dos diagramas UML permitiu estruturar de forma clara o funcionamento da Dashboard PicMoney, desde sua organização interna até a interação do usuário com os dados. A arquitetura em camadas contribuiu para separar responsabilidades, garantindo melhor manutenção, escalabilidade e reuso de código. Já os diagramas, Classes, Casos de Uso, Sequência e Atividades, possibilitaram visualizar tanto a estrutura do domínio quanto o fluxo de processamento e interação entre os componentes.

Com isso, foi possível compreender como os dados são armazenados, consultados e apresentados, além de evidenciar a lógica que sustenta as análises realizadas pelo CEO e CFO. Dessa forma, o trabalho não só documenta o sistema, mas também orienta futuras melhorias e evoluções da aplicação, assegurando que o desenvolvimento continue alinhado às necessidades de uso e às boas práticas de engenharia de software